

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1408

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration :60 Mins
Off Class
Total Marks:50

Annexure A

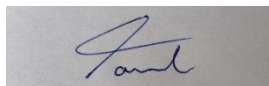
DESIGN TASK

Code of Conduct:

I Tamil Elakiya S (192524443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



1 . Design a parser to recognize simple grammar for palindromes.

Grammar

$S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow a$

$S \rightarrow b$

$S \rightarrow \epsilon$

Input

Abba

Design Requirements

(i) Implement the lexical analyzer using Lex.

(ii) Implement the parser.

(iii) Validate the palindrome string.

(iv) Display Accepted or Rejected.

(v) Handle syntax errors.

Aim :

To design and implement a parser using ANTLR to recognize palindrome strings consisting of characters a and b, validate whether the input satisfies the given grammar, display Accepted or Rejected, and handle syntax errors.

Design Procedure :

1. Create lexer rules to recognize the terminals a and b.
2. Create parser rules according to the palindrome grammar.
3. Generate lexer and parser using ANTLR.
4. Read the input string.
5. Parse the input.
6. If parsing completes successfully, display Accepted.
7. Otherwise display Rejected.

Code :

#antlr :

grammar Palindrome;

start : s EOF ;

s

: 'a' s 'a'

| 'b' s 'b'

| 'a'

| 'b'

|

;

WS : [\t\r\n]+ -> skip;

Java :

```
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.*;

public class Main {

    public static void main(String[] args) throws Exception {

        CharStream input = CharStreams.fromStream(System.in);

        PalindromeLexer lexer = new PalindromeLexer(input);

        CommonTokenStream tokens = new CommonTokenStream(lexer);

        PalindromeParser parser = new PalindromeParser(tokens);

        parser.removeErrorListeners();

        parser.addErrorListener(new BaseErrorListener() {

            @Override
            public void syntaxError(Recognizer, ? recognizer,
                                   Object offendingSymbol,
                                   int line,
                                   int charPositionInLine,
                                   String msg,
                                   RecognitionException e) {

                System.out.println("Rejected");
                System.exit(0);
            }

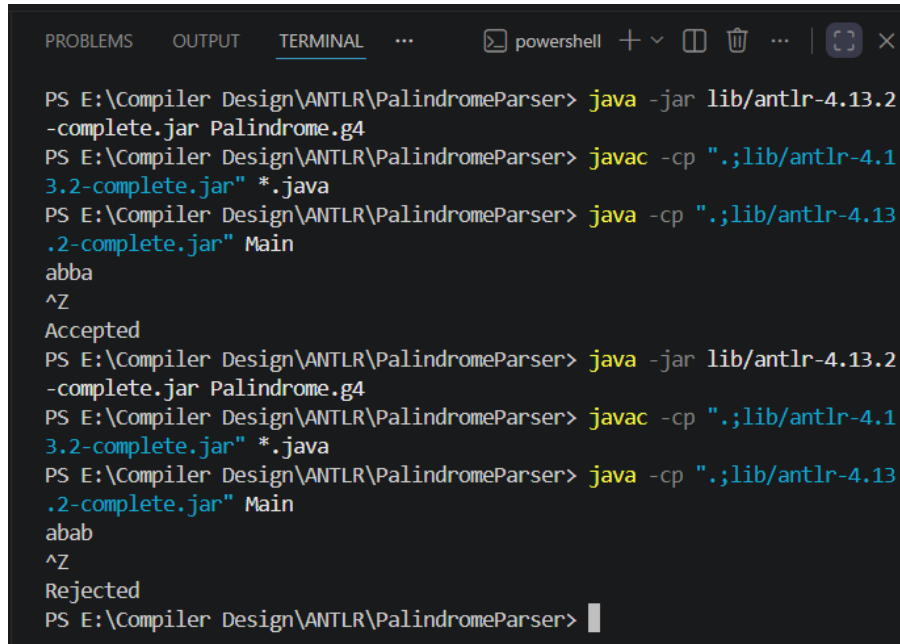
        });

        parser.start();

        System.out.println("Accepted");
    }

}
```

Output :



```
PS E:\Compiler Design\ANTLR\PalindromeParser> java -jar lib/antlr-4.13.2-complete.jar Palindrome.g4
PS E:\Compiler Design\ANTLR\PalindromeParser> javac -cp ".;lib/antlr-4.13.2-complete.jar" *.java
PS E:\Compiler Design\ANTLR\PalindromeParser> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
abba
^Z
Accepted
PS E:\Compiler Design\ANTLR\PalindromeParser> java -jar lib/antlr-4.13.2-complete.jar Palindrome.g4
PS E:\Compiler Design\ANTLR\PalindromeParser> javac -cp ".;lib/antlr-4.13.2-complete.jar" *.java
PS E:\Compiler Design\ANTLR\PalindromeParser> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
abab
^Z
Rejected
PS E:\Compiler Design\ANTLR\PalindromeParser> 
```

2. Design a parser to recognize balanced parentheses and brackets.

Grammar

$S \rightarrow (S)$

$S \rightarrow [S]$

$S \rightarrow SS$

$S \rightarrow \epsilon$

Input

(I()I)

Design Requirements

(i) Implement the lexical analyzer using Lex.

(ii) Implement the parser.

(iii) Validate the balanced symbols.

(iv) Display Accepted or Rejected.

(v) Handle syntax errors.

Aim

To design an ANTLR parser that recognizes balanced parentheses and brackets and validates properly nested expressions.

Design Procedure

1. Define lexer rules for '(' ')' '[' ']'.
2. Implement recursive grammar.
3. Generate parser.
4. Parse the input.
5. Print Accepted if valid.
6. Otherwise print Rejected.

Code :

#antlr :

```
grammar Balanced;
```

```
start : s EOF ;
```

```
s
    : '(' s ')' s
    | '[' s ']' s
    |
    ;
```

```
WS : [ \t\r\n]+ -> skip;
```

Java :

```
import org.antlr.v4.runtime.*;
```

```
public class Main {
```

```
    public static void main(String[] args) throws Exception {
```

```
        CharStream input = CharStreams.fromStream(System.in);
```

```
        BalancedLexer lexer = new BalancedLexer(input);
```

```
        CommonTokenStream tokens = new CommonTokenStream(lexer);
```

```
        BalancedParser parser = new BalancedParser(tokens);
```

```
        parser.removeErrorListeners();
```

```
        parser.addErrorListener(new BaseErrorListener(){
```

```
            @Override
```

```
            public void syntaxError(Recognizer<?,?> recognizer,
```

```
                Object offendingSymbol,
```

```
                int line,
```

```
                int charPositionInLine,
```

```
                String msg,
```

```
                RecognitionException e){
```

```
                System.out.println("Rejected");
```

```
                System.exit(0);
```

```

    }

});

parser.start();

System.out.println("Accepted");

}

}

```

Output :

```

PS E:\Compiler Design\ANTLR\BalancedParser>
PS E:\Compiler Design\ANTLR\BalancedParser>
PS E:\Compiler Design\ANTLR\BalancedParser> java -jar lib/antlr-4.13.2-complete.jar Balanced.g4
PS E:\Compiler Design\ANTLR\BalancedParser> javac -cp ".;lib/antlr-4.13.2-complete.jar" *.java
PS E:\Compiler Design\ANTLR\BalancedParser> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
([()])
Accepted
PS E:\Compiler Design\ANTLR\BalancedParser>

```

3 . Design a parser to recognize simple SQL WHERE clauses.

Grammar

Cond → id Relop value

Cond → Cond AND Cond

Cond → Cond OR Cond

Relop → = | < | >

Input

age>18 AND mark>75

Design Requirements

(i) Implement the lexical analyzer using Lex.

(ii) Implement the parser.

(iii) Validate the WHERE clause.

(iv) Display Accepted or Rejected.

(v) Handle syntax errors.

Aim

To design an ANTLR parser that recognizes simple SQL WHERE clauses consisting of relational operators connected using AND/OR.

Design Procedure

1. Define tokens for identifiers, numbers, strings, AND, OR.
2. Define parser rules.

3. Generate parser.
4. Parse user input.
5. Print Accepted if valid.
6. Print Rejected if syntax error occurs.

Code :

#antlr :

grammar SQLWhere;

start : cond EOF ;

cond
: cond AND cond
| cond OR cond
| ID relop value
;

relop
: '='
| '<'
| '>'
;

value
: NUMBER
| STRING
;

AND : 'AND';
OR : 'OR';

ID : [a-zA-Z_][a-zA-Z0-9_]*;

NUMBER : [0-9]+;

STRING : '"' .*? '"';

WS : [\t\r\n]+ -> skip;

Java :

import org.antlr.v4.runtime.*;

public class Main {

```

public static void main(String[] args) throws Exception {

    CharStream input = CharStreams.fromStream(System.in);

    SQLWhereLexer lexer = new SQLWhereLexer(input);

    CommonTokenStream tokens = new CommonTokenStream(lexer);

    SQLWhereParser parser = new SQLWhereParser(tokens);

    parser.removeErrorListeners();

    parser.addErrorListener(new BaseErrorListener(){

        @Override
        public void syntaxError(Recognizer,? recognizer,
                                Object offendingSymbol,
                                int line,
                                int charPositionInLine,
                                String msg,
                                RecognitionException e){

            System.out.println("Rejected");
            System.exit(0);

        }

    });

    parser.start();

    System.out.println("Accepted");

}

}

```

Output :

```

PS E:\Compiler Design\ANTLR\SQLParser> java -jar lib/antlr-4.13.2-complete.jar SQLWhere.g4
PS E:\Compiler Design\ANTLR\SQLParser> javac -cp ".;lib/antlr-4.13.2-complete.jar" *.java
PS E:\Compiler Design\ANTLR\SQLParser> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
age>18 AND mark>75
^Z
Accepted
PS E:\Compiler Design\ANTLR\SQLParser> █

```

4 . Design a parser to recognize JSON objects with nested arrays.

Grammar

Object → { PairList }

PairList → Pair

PairList → Pair , PairList

Pair → string : Array

Array → [ValueList]

ValueList → value

ValueList → value , ValueList

Input

{"marks":[90,85,95]}

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the JSON object.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim

To design an ANTLR parser that recognizes JSON objects containing arrays and validates the syntax.

Design Procedure

1. Create lexer rules for JSON symbols.
2. Define parser rules for object, pair, array, and values.
3. Generate parser.
4. Parse JSON input.
5. Print Accepted if valid.
6. Otherwise print Rejected.

Code :

antlr :

grammar Json;

start : object EOF;

object

: '{' pairList? '}'
;

pairList

: pair (',' pair)*
;

pair

```

    : STRING ':' array
    ;

array
    : '[' valueList? ']'
    ;

valueList
    : value (',' value)*
    ;

value
    : NUMBER
    | STRING
    | object
    | array
    ;

STRING : '"' (~["\\"] | '\\' .)* '"';

NUMBER : '-'? [0-9]+ ('.' [0-9]+)?;

WS : [ \t\r\n]+ -> skip;

```

Java :

```

import org.antlr.v4.runtime.*;

public class Main {

    public static void main(String[] args) throws Exception {

        CharStream input = CharStreams.fromStream(System.in);

        JsonLexer lexer = new JsonLexer(input);

        CommonTokenStream tokens = new CommonTokenStream(lexer);

        JsonParser parser = new JsonParser(tokens);

        parser.removeErrorListeners();

        parser.addErrorListener(new BaseErrorListener(){

```

```

@Override
public void syntaxError(Recognizer,? recognizer,
                        Object offendingSymbol,
                        int line,
                        int charPositionInLine,
                        String msg,
                        RecognitionException e){

    System.out.println("Rejected");
    System.exit(0);

}

});

parser.start();

System.out.println("Accepted");

}

}

```

Output :



The screenshot shows an IDE terminal window with the following tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal output is as follows:

```

PS E:\Compiler Design\ANTLR\JSONParser> java -jar lib/antlr-4.13.2-complete.jar Json.g4
PS E:\Compiler Design\ANTLR\JSONParser> javac -cp ".;lib/antlr-4.13.2-complete.jar" *.java
PS E:\Compiler Design\ANTLR\JSONParser> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
{"marks":[87,98,79,99]}

^Z
Accepted
PS E:\Compiler Design\ANTLR\JSONParser>

```

5 . Design a parser to recognize simple HTML documents with nested tags.

Grammar

Document → **Element**

Element → **<id> Content </id>**

Content → **Element**

Content → **string**

Content → ϵ

Input

<html>

<head></head>

<body>

<h1>Compiler Design</h1>

</body>

</html>

Design Requirements

(i) Implement the lexical analyzer using Lex.

(ii) Implement the parser.

(iii) Validate the HTML document structure.

(iv) Display Accepted or Rejected.

(v) Handle syntax errors.

Aim

To design an ANTLR parser that recognizes simple HTML documents with properly nested opening and closing tags.

Design Procedure

1. Create lexer rules for opening tags, closing tags, identifiers, and text.
2. Implement recursive grammar for nested elements.
3. Generate the parser using ANTLR.
4. Parse the HTML input.
5. Display Accepted if the structure is valid.
6. Display Rejected if a syntax error occurs.

Code :

antlr :

grammar Html;

start

: element+ EOF

;

element

: OPEN ID CLOSE content OPENS LASH ID CLOSE

;

content

```
: (element | TEXT)*  
;
```

OPEN

```
: '<'  
;
```

OPENSASH

```
: '</'  
;
```

CLOSE

```
: '>'  
;
```

ID

```
: [a-zA-Z][a-zA-Z0-9]*  
;
```

TEXT

```
: ~[<>]+  
;
```

WS

```
: [ \t\r\n]+ -> skip ;
```

Java :

```
import org.antlr.v4.runtime.*;  
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) throws Exception {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.println("Enter HTML:");
```

```
        String input = sc.nextLine();
```

```
        CharStream cs = CharStreams.fromString(input);
```

```
        HtmlLexer lexer = new HtmlLexer(cs);
```

```
        CommonTokenStream tokens = new CommonTokenStream(lexer);
```

```

HtmlParser parser = new HtmlParser(tokens);

parser.removeErrorListeners();

parser.addErrorListener(new BaseErrorListener(){

    @Override
    public void syntaxError(
        Recognizer<?,?> recognizer,
        Object offendingSymbol,
        int line,
        int charPositionInLine,
        String msg,
        RecognitionException e){

        System.out.println("Rejected");

        System.exit(0);

    }

});

HtmlParser.StartContext tree = parser.start();

HtmlCheckVisitor visitor = new HtmlCheckVisitor();

boolean ok = true;

for(HtmlParser.ElementContext e : tree.element()){

    ok &= visitor.visit(e);

}

if(ok)

    System.out.println("Accepted");

else

    System.out.println("Rejected");

}

}

```

Output :

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS E:\Compiler Design\ANTLR\HTMLParser> java -jar lib/antlr-4.13.2-complete.jar -visitor Html.g4
PS E:\Compiler Design\ANTLR\HTMLParser> java -jar lib/antlr-4.13.2-complete.jar -visitor Html.g4
PS E:\Compiler Design\ANTLR\HTMLParser> javac -cp ".;lib/antlr-4.13.2-complete.jar" *.java
PS E:\Compiler Design\ANTLR\HTMLParser> java -cp ".;lib/antlr-4.13.2-complete.jar" Main
Enter HTML:
<html><body><h1>Compiler Design</h1></body></html>
Accepted
PS E:\Compiler Design\ANTLR\HTMLParser> █
```